

ServiceNow Script-Controlled ACL Project

Restricting record access based on field values and user conditions

Team ID: SWTID-2026-3899

Team Size: 5 Members

Platform: ServiceNow PDI

Framework: SmartBridge Phase-Wise Delivery

PROJECT TEAM STRUCTURE

AKASH D

Team Leader

I. DHIVAKAR

Team Member

KAVIN.R

Team Member

VEERRAMANI.G

Team Member

SRNIVASAN

Team Member

01. Brainstorming & Ideation Phase

- **Objective:** Address security gaps in standard ServiceNow role-based access controls by engineering scriptable Access Control Lists (ACLs) that enforce dynamic, context-aware record access.
- **Problem Statement:** Standard role-based security in ServiceNow grants broad access once a role is assigned. It fails to restrict access based on dynamic record states (e.g., urgency, classification, sensitive data flags) or specific user context (e.g., manager hierarchy, assignment group membership, location).
- **Proposed Solution:** Implement advanced Server-Side Access Control Lists (ACLs) using JavaScript (current and gs objects) to dynamically evaluate access grants across read, write, create, and delete operations.
- **Core Use Cases:**
 - Restricting record editing permissions based on assignment group membership and priority level.
 - Dynamically concealing confidential or sensitive fields unless the user possesses elevated security credentials or belongs to a specific department.
 - Restricting record deletion to record owners or system administrators.

02. Requirement Analysis Phase

- **Functional Requirements:**

- Define target tables (e.g., Incident, Task, or custom security tables) for script-controlled ACL implementation.
- Implement custom JavaScript logic within ACL records evaluating both table-level and field-level permissions.
- Incorporate session checks via the GlideSystem API (`gs.getUser()`, `gs.getUserID()`, role checks, and group queries).
- Ensure separation of concerns between standard role requirements and dynamic script evaluations.

- **Non-Functional Requirements:**

- **Performance:** Optimize ACL scripts to avoid heavy queries (such as unindexed `GlideRecord` loops) to maintain low query latency on form and list views.
- **Security:** Follow ServiceNow security best practices, ensuring scripts fail securely (`answer = false` by default).
- **Maintainability:** Standardize variable naming and maintain clean, well-commented code blocks across all ACL scripts.

03. Project Design Phase

- **Architecture & Components:**

- **ACL Definitions (`sys_security_acl`):** Configured for specific record operations (`record/read`, `record/write`, `record/create`, `record/delete`).
- **Script Logic Architecture:** Utilize standard ServiceNow ACL script evaluation structure setting the `answer` variable (`answer = true;` or `answer = false;`).
- **Core APIs & Objects:**
 - `current`: Reference to the active record being accessed or evaluated.
 - `gs`: `GlideSystem` object for session details, current user context, and logging.

- **Access Control Evaluation Workflow:** Role Check → Condition Evaluation → Script Execution (returns `answer`) → Grant/Deny Access.

04. Project Planning Phase

Phase / Module	Assigned Members	Deliverables
Ideation & Requirements	AKASH D (TL), I.DHIVAKAR	Problem statement, functional requirements doc
Design & Planning	KAVIN.R, VEERRAMANI.G	ACL architecture flow, field mapping matrix
Development	AKASH D, I.DHIVAKAR, SRNIVASAN	Servicenow PDI script configuration & ACL rules
Testing & QA	KAVIN.R, VEERRAMANI.G	Security debug traces & role impersonation logs
Documentation & Demo	Entire Team	Public GitHub Repository & Demo Video

05. Project Development Phase

Implementation Steps & Server-Side Scripts:

1. Table-Level Write Access Script:

```
// Restrict write access to assignment group members or admins
answer = false;
if (gs.getUser().hasRole('admin') || current.assignment_group.isMember(gs.getUserID())) {
    answer = true;
} else if (current.getValue('u_sensitivity_level') == 'Low') {
    answer = true;
}
```

2. Field-Level Read Access Script:

```
// Restrict view access to confidential field based on user department
answer = false;
var userDept = gs.getUser().getDepartment();
if (userDept == 'HR' || current.opened_by == gs.getUserID()) {
    answer = true;
}
```

06. Project Testing Phase

- **Test Case 1 (Unauthorized Edit):** Standard user attempts to modify a high-sensitivity record → *Expected Result:* Access Denied / Fields remain read-only.
- **Test Case 2 (Authorized Edit):** Assigned group member accesses the same record → *Expected Result:* Access Granted / Edit enabled.
- **Test Case 3 (Field Hiding):** Non-HR user views record containing HR-sensitive fields → *Expected Result:* Fields hidden from layout.
- **Debugging Strategy:** Utilize ServiceNow System Logs (gs.log()) and Security Debugging tools (Debug Security) to trace ACL rule evaluation.

07. Project Documentation Phase

GitHub Repository Structure:

- /01-ideation — Problem statement & scope notes
- /02-requirements — Functional specs & access matrices
- /03-design — Architectural flowcharts
- /04-planning — Task assignment logs
- /05-development — ACL XML exports & script snippets
- /06-testing — Test scripts & security debug traces
- /07-documentation — Operational manual
- README.md — Project landing page

08. Project Demonstration Phase

- **Demo Video Presentation Checklist:**
 - Team introduction and Team ID display (**SWTID-2026-3899**).
 - Explanation of project purpose and enterprise security benefits.
 - Screen recording in ServiceNow PDI walking through ACL script rules.
 - Live execution testing using user impersonation.